SISTEMAS OPERATIVOS · EAFIT · 2026-2
EVALUACIÓN PARCIAL PRÁCTICA-TEÓRICA · 25%

Seguridad de Archivos mediante Intervención en la Capa del Kernel y mmap

mmapguard — mapeo protegido con descifrado bajo demanda

Autor · Sebastián Martínez Maya

Modalidad · individual

Entregable · módulo de kernel, herramientas, banco de pruebas e informe

Kernel de desarrollo · 6.18.33.2-microsoft-standard-WSL2, x86_64

Banco de pruebas · 8 de 8 PoC superadas

CONTENIDO

PARTE I · 30%

Análisis conceptual: diez preguntas · 2

PARTE II · 30%

Flujo de trabajo asistido por IA · 5

Ingeniería de prompts y verificación crítica · 7

Estrategia arquitectónica seleccionada · 10

PARTE III · 40%

Implementación: código, compilación y PoC · 13

Medición de impacto y análisis de resultados · 15

Justificación: kernel frente a espacio de usuario · 18

Generado el 10/11/2026 · las citas archivo:línea remiten al árbol del kernel 6.18.33.2

Parte I — Análisis Conceptual y Preguntas Diagnósticas (30%)

> Referencias al árbol `linux-msft-wsl-6.18.33.2`, que es el kernel en ejecución (`6.18.33.2-microsoft-standard-WSL2`, x86_64, `CONFIG_PGTABLE_LEVELS=5`). Cada afirmación verificable lleva archivo y línea.

1. Mapeo virtual y Page Faults: secuencia exacta

Son dos fases con actores distintos: establecer el mapeo es software puro en Ring 0; materializar las páginas es una excepción de hardware.

Fase A — alta del VMA, una sola vez. `SYSCALL_DEFINE6(mmap, …)` (`arch/x86/kernel/sys_x86_64.c:82`) → `ksys_mmap_pgoff()` (`mm/mmap.c:566`) → `do_mmap()` (`mm/mmap.c:334`), que resuelve el `fd` a `struct file *`, valida longitud, alineación y `RLIMIT_AS` e invoca el gancho LSM `security_mmap_file()` (`include/linux/security.h:479`). `get_unmapped_area()` localiza un hueco, se asigna un `struct vm_area_struct` y se rellenan sus campos, entre ellos `vm_page_prot` con `vma_set_page_prot()` (`mm/mmap.c:81`), que traduce las banderas del VMA a los bits que acabarán en la PTE. Por último `vfs_mmap()` (`include/linux/fs.h:2403`) despacha a `f_op->mmap()`, donde el driver instala su `vma->vm_ops`. **Al retornar no existe ni una sola PTE ni se ha reservado un marco físico.**

Fase B — primer acceso. La MMU recorre los cinco niveles desde CR3 y encuentra `_PAGE_BIT_PRESENT` en cero. La CPU lanza el vector 14 con la dirección en CR2 y un código de error que distingue lectura de escritura y usuario de supervisor. `DEFINE_IDTENTRY_RAW_ERRORCODE(exc_page_fault)` (`arch/x86/mm/fault.c:1483`) deriva en `do_user_addr_fault()` (`:1207`), que intenta la vía rápida sin `mmap_lock` con `lock_vma_under_rcu()` (`:1325`). Sigue `handle_mm_fault()` (`mm/memory.c:6471`) → `handle_pte_fault()` (`:6152`) → `do_fault()` (`:5821`), que elige entre `do_read_fault()`, `do_cow_fault()` y `do_shared_fault()` (`:5771`). Se invoca `vma->vm_ops->fault`, se instala la PTE, se invalida el TLB y **la CPU reejecuta la instrucción**, que ahora se resuelve íntegramente por hardware.

Desde ese punto, **todo acceso posterior a esa página ocurre sin intervención del kernel**. Es la propiedad que da su rendimiento a `mmap()` y, a la vez, la que crea el problema de este parcial.

2. `MAP_SHARED` frente a `MAP_PRIVATE`

`MAP_SHARED` activa `VM_SHARED | VM_MAYSHARE`: los fallos de escritura se atienden con `do_shared_fault()`, que trabaja sobre el folio de la Page Cache del inodo, de modo que lo escrito es visible de inmediato para cualquier otro proceso y el writeback acabará persistiéndolo. `MAP_PRIVATE` entra en `do_cow_fault()`, que crea una copia anónima: no hay persistencia ni visibilidad cruzada.

Un detalle del código resultó decisivo para este trabajo. En la rama `MAP_SHARED` de `do_mmap()` (`mm/mmap.c:462-464`):

```
vm_flags |= VM_SHARED | VM_MAYSHARE;
if (!(file->f_mode & FMODE_WRITE))
        vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
```

Si el descriptor no es escribible, el kernel **degrada el mapeo en silencio**: no falla, cambia la semántica pedida. Un filtro que razone sobre las banderas que pidió el usuario, y no sobre las que quedaron en el VMA, analiza algo que el kernel no construyó. Esto se verificó en la práctica (PoC A2) y obligó a corregir la prueba, no el módulo.

Seguridad. Con `MAP_SHARED`, un mapeo escribible da persistencia y visibilidad cruzada **sin ejecutar una sola syscall de escritura**, de modo que un auditor que vigile `write()` queda ciego. Con `MAP_PRIVATE` no hay persistencia, pero sí una copia completa en claro en el espacio del atacante: se pierde confidencialidad aunque se conserve la integridad.

3. Copy-On-Write: ciclo de vida y escalada

Tras `fork()`, o en un mapeo privado de archivo, no se duplica nada: ambas PTE se marcan solo lectura limpiando `_PAGE_BIT_RW`, mientras el VMA conserva `VM_WRITE` y `VM_MAYWRITE`. Esa discrepancia deliberada entre «el VMA permite escribir» y «la PTE no deja» es el disparador. Al escribir, el #PF deriva en `do_wp_page()` (`mm/memory.c:4049`): si el folio es exclusivo del proceso —`wp_can_reuse_anon_folio()` (`mm/memory.c:3986`)— basta reactivar `_PAGE_BIT_RW`; si no, `wp_page_copy()` (`mm/memory.c:3658`) reserva un folio nuevo y reapunta la PTE.

Dirty COW (CVE-2016-5195) no fue un fallo de permisos sino de atomicidad: `PROT_WRITE`, `VM_MAYWRITE` y los bits de la PTE eran correctos en todo momento. El atacante mapeaba `MAP_PRIVATE` un archivo de root legible y escribía por `/proc/self/mem` mientras un hilo concurrente lanzaba `madvise(MADV_DONTNEED)` sobre el rango, descartando la copia privada recién creada; al reintentar, `__get_user_pages()` abandonaba la exigencia de escritura privada y terminaba escribiendo **sobre el folio de la Page Cache del archivo original**.

La corrección no añadió un candado: separó semánticamente «escribir» de «romper el compartido». `FAULT_FLAG_UNSHARE` (`include/linux/mm_types.h:1659`) hizo de la ruptura de COW una operación explícita e idempotente, y combinarla con `FAULT_FLAG_WRITE` se declaró ilegal (`:1644`).

Lección que gobierna este diseño: un control que se evalúa una vez y cuyo resultado se reutiliza es un objetivo de carrera.

4. Intercepción: hook de la syscall table frente a LSM

Corrección de nomenclatura. El enunciado menciona el gancho `file_mmap`. **No existe en los kernels modernos.** El real es `mmap_file` (`include/linux/lsm_hook_defs.h:199`), invocado por `security_mmap_file()` (`include/linux/security.h:479`). `file_mmap` es nomenclatura anterior a la serie 3.x que sobrevive en tutoriales y en la salida de los asistentes de IA, y por eso es un buen indicador de material desactualizado.

LKM con hook en la syscall table. Da control total y cobertura global, pero tiene cuatro obstáculos y cada uno basta para descartarlo: (1) `sys_call_table` no está exportada y `kallsyms_lookup_name()` tampoco desde 5.7; (2) escribir la tabla obliga a limpiar el bit WP de `CR0`, desarmando una protección del procesador para todo el sistema; (3) la reescritura no es serializable —otra CPU puede estar despachando esa misma syscall—; (4) en x86_64 el despacho pasa por envoltorios `__x64_sys_*(struct pt_regs *)` y la técnica choca con CFI y con `lockdown`.

LSM. Es la interfaz diseñada para esto: estable, situada tras resolver el descriptor y antes de crear el VMA, con denegación por valor de retorno. **Desventaja decisiva:** `security_add_hooks()` no está exportada y la tabla se fija en compilación, así que **no se puede registrar un LSM desde un módulo cargable**.

La vía practicable hoy es **BPF LSM** (pregunta 7).

5. Cifrado transparente en memoria

La respuesta en árbol es **fscrypt**: el sistema de archivos cifra por bloques en el writeback y descifra en la lectura con `fscrypt_decrypt_pagecache_blocks()` (`include/linux/fscrypt.h:353`), con clave por inodo derivada de una maestra en el llavero del kernel. Para que `mmap()` sea legible no hace falta nada especial: `filemap_fault` entrega el folio de la Page Cache, que **ya contiene texto en claro** porque el descifrado ocurrió al poblarla.

Y ahí está el punto. **La Page Cache almacena texto en claro**, de modo que el aislamiento no lo aporta la criptografía sino los permisos ordinarios del VFS: cualquier proceso autorizado a abrir el archivo ve el contenido descifrado, y dos procesos que lo mapeen comparten los mismos marcos físicos. fscrypt protege **datos en reposo** —un disco sustraído, un respaldo filtrado— y no frente a un proceso local no privilegiado, que es justamente el atacante de nuestro modelo.

6. `msync`, `fsync` y páginas sucias

El kernel mantiene la suciedad en dos sitios, y confundirlos es la fuente habitual de error: el **bit de hardware** `_PAGE_BIT_DIRTY` de la PTE (bit 6, `arch/x86/include/asm/pgtable_types.h:16`), que activa la CPU sola al escribir, y el **bit de software** `PG_dirty` del folio, que es el que consulta el writeback.

Para un `MAP_SHARED` de archivo el kernel **no concede la escritura directamente**: mapea de solo lectura a propósito para que la primera escritura genere una trampa. `do_shared_fault()` (`mm/memory.c:5771`) llama a `do_page_mkwrite()` (`mm/memory.c:3478`), que invoca `vm_ops->page_mkwrite()` (`mm/memory.c:3489`); el sistema de archivos reserva bloques y marca el folio sucio, y **solo entonces** se activa `_PAGE_BIT_RW`. A partir de ahí toda escritura es hardware puro. `msync(MS_SYNC)` traslada el estado sucio a los folios e inicia el writeback.

Cómo se evade un filtro. `page_mkwrite` se dispara **una sola vez por página**. Un filtro que inspeccione el contenido en ese instante y deje la página escribible implementa un TOCTOU de manual: el atacante escribe contenido inocuo, supera la inspección y sobrescribe con la carga maliciosa **sin que el kernel se entere**, porque ya no hay fallos que atender. Solo quedan dos puntos de inspección sólidos: en el writeback, barato pero tardío, o manteniendo la página de solo lectura y aceptando un fallo por escritura, inmediato pero anulando la ventaja de `mmap()`.

`mmapguard` disuelve el dilema en vez de escoger un lado (sección 3.3): el mapeo es de solo lectura, así que no existe una primera escritura que inspeccionar y la decisión se toma en el alta del VMA. El precio es la funcionalidad, y el día que se admitan modificaciones el dilema reaparece intacto.

7. eBPF, kprobes y fexit

Tipo	Punto de enganche	¿Puede denegar?
`kprobe` / `kretprobe`	Cualquier símbolo del kernel	No
`fentry` / `fexit`	Entrada/salida de función, vía BTF	No
Tracepoints	Puntos estáticos de ABI estable	No
BPF LSM (`SEC("lsm/...")`)	Ganchos del framework LSM	**Sí**, por valor de retorno

La distinción decide la arquitectura: `fentry` y `kprobe` **observan pero no alteran el flujo**, su valor de retorno se ignora. Solo BPF LSM deniega, porque se inserta en una cadena cuyo contrato es exactamente ése. No hace falta recompilar por **CO-RE** apoyado en BTF, disponible aquí (`CONFIG_DEBUG_INFO_BTF=y`): el kernel publica su descripción de tipos y libbpf reubica los desplazamientos en carga.

Condición de arranque aquí: `CONFIG_BPF_LSM=y`, pero `CONFIG_LSM` no incluye `bpf` y `/proc/cmdline` no trae `lsm=`. Los ganchos están compilados pero **no son adjuntables**, que es por lo que el plano 2 queda documentado y no implementado.

8. Evasión de permisos POSIX

Sí, el kernel lo permite. La asimetría entre las dos ramas de `do_mmap()` es explícita. Rama `MAP_SHARED`:

```
if (prot & PROT_WRITE) {
        if (!(file->f_mode & FMODE_WRITE))
                return -EACCES;
```

Rama `MAP_PRIVATE`, en su totalidad en lo que a permisos respecta:

```
case MAP_PRIVATE:
        if (!(file->f_mode & FMODE_READ))
                return -EACCES;
```

No hay chequeo de escritura: solo se exige poder leer. Es coherente con la semántica de `MAP_PRIVATE` —toda escritura se desvía por COW y el archivo nunca se modifica— y equivale a leer a un búfer con `read()` y modificarlo, que nadie consideraría una violación de permisos.

Riesgos. El contenido queda en claro y modificable en el espacio del atacante, de modo que cualquier control de integridad que inspeccione el archivo es irrelevante para lo que el proceso realmente usa; las páginas privadas modificadas pueden acabar en swap o en un volcado, razón por la que el plano 1 fija `VM_DONTDUMP`; y es exactamente la configuración que explotó Dirty COW. Verificado en la PoC A4 (sección 4.2): **la política no puede asumir que `O_RDONLY` equivale a inofensivo.**

9. Carreras entre `madvise(MADV_DONTNEED)` y escrituras

`madvise_dontneed_free()` (`mm/madvise.c:907`) termina en `zap_page_range_single()` (`:1187`), que **desmonta las PTE del rango**: para una página privada bajo COW descarta la copia anónima y el siguiente acceso vuelve a fallar. Permite al atacante **revertir el estado de una página a voluntad desde espacio de usuario**, y eso es lo que la hace una primitiva de carrera tan útil.

El kernel actual serializa con cuatro mecanismos: el `mmap_lock`; el bloqueo por VMA (`lock_vma_under_rcu()`, `arch/x86/mm/fault.c:1325`), cuya exclusión se obtiene con `vma_start_write()` —que es lo que invoca `vm_flags_set()` internamente (`include/linux/mm.h:808`), y **la razón de fondo por la que se prohibió asignar directamente a `vma->vm_flags`**—; el spinlock de la tabla de páginas vía `pte_offset_map_lock()`; y la invalidación de TLB con IPI antes de liberar el marco.

Ninguno faltaba en 2016: el fallo era de **lógica de reintento**, y la corrección cambió el modelo en vez de añadir un candado (pregunta 3).

Principio aplicable a nuestra defensa: los controles deben ser idempotentes y reevaluarse en cada fallo, nunca cachearse como «ya validado». Un veredicto almacenado es un objetivo de carrera, y la PoC A5 lo comprueba: sin escritura posible no hay precondición de carrera.

10. Aislamiento: descriptores, tablas y bits de la MMU

`struct page` / `struct folio` es el descriptor por marco físico —referencias, `mapping`, banderas como `PG_dirty`—, metadato del kernel jamás direccionable desde Ring 3. La contención empieza ahí: el proceso manipula direcciones virtuales, nunca descriptores de marco. **Jerarquía de tablas:** con `CONFIG_PGTABLE_LEVELS=5`, PGD → P4D → PUD → PMD → PTE, recorrida por hardware desde CR3. Cada proceso tiene su propia raíz, y ahí reside el aislamiento fundamental: una dirección virtual del proceso A **no tiene traducción** hacia los marcos del proceso B.

Bits de protección (`arch/x86/include/asm/pgtable_types.h`):

Bit	Nombre	Pos.	Función en la contención
P	`_PAGE_BIT_PRESENT`	0	Si es 0, cualquier acceso genera #PF: base de la paginación por demanda
R/W	`_PAGE_BIT_RW`	1	Soporte de COW y de la protección contra escritura
U/S	`_PAGE_BIT_USER`	2	Separa Ring 3 de Ring 0
D	`_PAGE_BIT_DIRTY`	6	Lo activa la CPU al escribir
NX	`_PAGE_BIT_NX`	63	Impide ejecutar la página

NX materializa W^X y volvió inviable la inyección clásica de shellcode, desplazando la explotación hacia ROP. **U/S**, con SMEP y SMAP, cierra el ataque de hacer que el kernel salte a una carga en espacio de usuario, y SMAP es la razón de hardware por la que un driver usa `copy_to_user()` y no `memcpy()`. **R/W** sostiene COW, y la pregunta 3 demuestra que los bits por sí solos no bastan: deben evaluarse atómicamente respecto al acceso.

El límite que justifica este trabajo. Los bits de la MMU se aplican **por traducción, no por contenido**: protegen rutas de acceso —quién puede leer, escribir o ejecutar— y **nada en la MMU puede decir si los bytes dentro de una página legítimamente escribible son maliciosos**, ni si quien escribe está autorizado por una política de aplicación. Esa brecha es la que cubren los planos 1 y 3 de la arquitectura propuesta.

3.1 Definición del Flujo de Trabajo Asistido por IA

1. Premisa: la asimetría que define el método

El trabajo se desarrolló con un modelo de lenguaje con acceso a terminal (Claude Code) sobre la máquina de desarrollo. Esa capacidad —ejecutar comandos, no solo redactar— determina la forma del flujo por una razón concreta:

> El modelo tiene conocimiento amplio del kernel de Linux pero **difuso en la versión**. El árbol de fuentes de la versión exacta está en el disco, a un `grep` de distancia.

Un modelo entrenado sobre dos décadas de código de kernel no distingue con fiabilidad qué era cierto en 5.10, qué en 6.1 y qué en 6.18: produce un promedio histórico, idiomático y convincente, sesgado hacia lo antiguo porque lleva más tiempo acumulándose. De ahí la regla operativa, establecida una sola vez y vigente toda la sesión:

> **Antes de afirmar que una API existe, verificarlo contra `kernel-wsl/` y citar archivo y línea.**

No es una regla de estilo: convierte una afirmación de memoria, que no se puede evaluar, en una afirmación con una referencia que el lector puede abrir y desmentir.

2. División del trabajo

Responsabilidad	Quién
Alcance, plazo y criterio de «suficiente»	Humano
Elección del mapeo de solo lectura (Opción A)	Humano, sobre consecuencias expuestas por el modelo
Qué planos se comprometen y cuál queda condicional	Humano
Enumeración de mecanismos viables y descarte razonado	Modelo
Redacción del código C y de las herramientas	Modelo
Verificación de APIs contra el árbol de fuentes	Modelo, con el árbol como oráculo
Ejecución de lo privilegiado: `insmod`, `rmmod`, `chmod` en `/dev`	Humano
Reporte de los fallos reales observados al cargar	Humano
Diseño de la medición y atribución del sobrecosto	Modelo

Las dos últimas filas humanas no son triviales: el problema más difícil de la sesión —el desajuste de CRC de `MODVERSIONS`— solo se pudo diagnosticar porque el usuario ejecutó `insmod` con privilegios y trajo el mensaje exacto de `dmesg`. Sin ese circuito de retorno el síntoma habría sido invisible.

3. El flujo en cuatro fases

Lo importante no es el orden —que es obvio— sino que ninguna fase avanza sobre afirmaciones no verificadas de la anterior.

Fase	Actividad	Criterio de salida
0	Reconocimiento: qué pide el parcial, qué entorno hay, qué es reutilizable	Un árbol de kernel que produce módulos **cargables**, verificado con un módulo trivial
1	Exploración arquitectónica: enumerar mecanismos, descartar con razón, asignar un plano por objetivo	Cada descarte respaldado por una comprobación contra el árbol, no por una intuición
2	Generación del código C, **con verificación API por API antes de compilar**	Compilación sin una advertencia con `-Wall -Wextra` y carga efectiva
3	PoC de los vectores de ataque y medición del sobrecosto	Cifras reproducibles, con la causa del sobrecosto atribuida y demostrada

La fase 0 terminó mucho más tarde de lo previsto, y ese desfase es el dato metodológico más útil del proyecto: entre tener el árbol y tener un árbol que produce módulos cargables hubo dos problemas no evidentes —el sufijo `+` de versión local y el CRC de `MODVERSIONS`—. Cualquier planificación que dé la fase 0 por hecha subestima el proyecto.

4. Rol de la IA en la exploración arquitectónica

El valor no estuvo en proponer una solución, sino en producir el conjunto de descartes. Pedir «cómo protejo archivos mapeados» devuelve una respuesta; pedir «enumera los mecanismos, descarta los que no sirven y di por qué» devuelve cuatro rutas cerradas, cada una con la razón que la cierra:

Mecanismo	Razón del descarte
Hook en la tabla de syscalls	Exige desarmar el bit WP de `CR0`; frágil y detectable

Mecanismo	Razón del descarte
LSM propio como módulo cargable	`security_add_hooks()` no está exportada: no hay vía legítima
eBPF solo con `fentry` / `kprobes`	Observa pero **no puede denegar**: su retorno se ignora
`remap_pfn_range()` (lo del material de referencia)	Obliga a mantener el archivo **completo** descifrado en RAM

Un descarte razonado es defendible ante una pregunta; una elección sin alternativas, no.

Un caso en que la exploración cambió el diseño, no solo lo justificó. El descarte de `remap_pfn_range()` empezó como comparación de rendimiento y terminó siendo de seguridad: si las páginas se entregan de a una bajo demanda, la ventana de exposición en memoria física baja de «todo el archivo» a «las páginas tocadas». La paginación por demanda, que en el material de referencia es una optimización, se convirtió en un **control de minimización de exposición**.

Un caso en que evitó trabajo inútil. Antes de escribir una línea de eBPF se verificó la viabilidad real: `CONFIG_BPF_LSM=y`, pero `CONFIG_LSM` no incluye `bpf`. Los ganchos existen y **no son adjuntables**. Un programa BPF LSM habría compilado, habría pasado el verificador y habría fallado al adjuntarse. El plano 2 se declaró condicional con esa evidencia, no por falta de tiempo.

Y el caso en contra, que importa igual. En esta misma fase el modelo estuvo a punto de hacer descartar la ruta del módulo cargable —el 40 % de la nota— por una lectura truncada de un bloque de `Kconfig` (alucinación A3 de 3.2). La exploración arquitectónica es donde una conclusión equivocada sale más cara, porque no la contradice ningún compilador. De ahí que el criterio de salida de la fase 1 sea una comprobación, no un consenso.

5. Rol de la IA en el código C de bajo nivel

> **Generar y verificar son dos actividades con fiabilidad muy distinta, aunque las ejecute el mismo modelo.** La primera produjo código plausible con cinco APIs inexistentes. La segunda, con el árbol como oráculo, las encontró todas antes de compilar.

Afirmación generada	Realidad en 6.18.33.2
`.llseek = no_llseek`	Eliminado del kernel en 6.12
`#include <asm/unaligned.h>`	Se mudó a `<linux/unaligned.h>`
`vma->vm_flags |= ...`	`vm_flags` es `const` en una unión: hay que usar `vm_flags_set/clear`
`class_create(THIS_MODULE, name)`	Recibe un solo argumento desde 6.4
`kallsyms_lookup_name()`	Ya no está exportada

Ninguna de las cinco es un error de razonamiento: todas fueron correctas y están en innumerables ejemplos publicados. El modo de fallo no es «el modelo se inventa cosas», es **el modelo escribe código correcto para una versión que no es la nuestra**. Consecuencia sobre el flujo: la generación se trató como borrador, nunca como entregable, y entre generar y compilar se interpuso un paso obligatorio de verificación símbolo por símbolo. No basta con dejar que filtre el compilador: este filtra sintaxis y tipos, no errores de **versión, configuración y entorno**, que en kernel son los que cuestan el tiempo —de ahí que el oráculo del flujo sea el árbol de fuentes y no el `make`.

6. Rol de la IA en las PoC controladas

El sesgo a vencer aquí es más incómodo: **el modelo tiende a escribir pruebas que pasan y a reportar cifras favorables.** Tres contramedidas explícitas:

(a) Blindar la medición contra el optimizador. No fue una exigencia teórica: salió de encontrar el fallo en el banco del repositorio de referencia (`paso5_benchmark_comparativo`), que compila con `-O2` y nunca consume el checksum del bucle de `mmap`, así que GCC lo elimina por código muerto: reporta **3.134.335 MB/s y 0 fallos de página** para 64 MB —ambas físicamente imposibles— y concluye que `mmap` es 380 veces más rápido; con `-O0`, 11.552 MB/s y ventaja real ~1,5x. El código está bien escrito: el fallo está en la **metodología de medición**, que es el patrón que un modelo reproduce con más facilidad porque es el que abunda.

(b) Rechazar la cifra única. La primera medición de latencia de `mmap()` dio 43,2 µs frente a 3,2 µs, un factor de 13 que invitaba a un titular. Repetida once veces y tomada la mediana, la diferencia quedó bajo el ruido del planificador, y se reportó el resultado negativo.

(c) No esconder el sobrecosto, atribuirlo. El resultado honesto es **17,60x** en acceso secuencial. La aportación no fue maquillarlo sino explicarlo —el costo dominante no es AES sino la ausencia de fault-around, y la sección 4.2 lo descompone—, y de ahí salió una propuesta de mejora cuantificada que una cifra maquillada no habría permitido formular. De las seis PoC diseñadas se ejecutaron las seis que aplican (A0–A5, 8/8 PASS); A6 exige AEAD y se declara no aplicable en vez de presentarse como verificada.

7. Lo que este flujo no cubre

Detecta afirmaciones falsas, no afirmaciones no verificadas que resultan ciertas. La alucinación A5 de 3.2 acertó, y precisamente por eso no la detectó nada: una afirmación causal confiada y correcta es, para quien lee, indistinguible de una verificada. El flujo reduce la probabilidad de error, no la de exceso de confianza. **Verifica el presente, no el futuro:** `no_llseek` también tenía su archivo y su línea en 6.11, así que toda la verificación de este informe caduca con la versión. Y **depende de que exista un oráculo**: donde no lo hay —decisiones de diseño, juicios de suficiencia, compensaciones— el flujo no aporta nada y la responsabilidad vuelve entera al humano. La elección del mapeo de solo lectura es de ese tipo: el modelo expuso la consecuencia criptográfica de reutilizar el nonce en CTR, pero aceptar perder la escritura a cambio de un cifrado correcto fue una decisión de alcance, no un resultado verificable.

3.2 Metodología de Ingeniería de Prompts y Verificación Crítica

1. Naturaleza del registro

La solución se desarrolló en una sesión asistida por un modelo con acceso a terminal (Claude Code), de forma **conversacional y no mediante prompts pre-redactados**. Los prompts de la sección 2 son literales; la descomposición en rol y restricción es un análisis posterior. Se aclara porque la alternativa —redactar a posteriori prompts elaborados que nunca se usaron— produciría una tabla más vistosa y completamente falsa.

2. Tabla de prompts estructurados

#	Prompt (literal)	Rol asignado	Restricción	Resultado esperado
P1	«clona este repo … con lo que contiene lo puedes correr para verificar la función mmap … pero si algo sale mal avísame»	Verificador autónomo	Reportar fallos en vez de ocultarlos	Diagnóstico del estado real del repositorio
P2	«en este pdf está toda la información … por ahora revisa todo el reporte»	Analista de requisitos	No actuar todavía, solo analizar	Desglose de entregables, pesos y bloqueos
P3	«hagamos la arquitectura como recomiendas»	Arquitecto de sistemas	Justificar cada descarte, no solo la elección	Documento 3.3 con matriz de compensaciones
P4	«arranca con las 10 preguntas»	Experto en kernel Linux	Fundamentar contra el árbol real, citando archivo y línea	10 respuestas verificables
P5	«opción A, empieza con el mmapguard.ko»	Desarrollador de kernel	Solo lectura; sin advertencias de compilación	Módulo funcional y cargable
P6	«sigue con la medición del plano 3 que vale el 40 %»	Ingeniero de rendimiento	Medición rigurosa, no cifras vistosas	Benchmark con metodología defendible

3. Alucinaciones técnicas detectadas

Las cinco primeras se detectaron **antes de compilar o antes de cargar**, por inspección del árbol o por contradicción empírica. Las tres últimas —A6, A7 y A8— las encontró el banco de pruebas al ejecutarse, y por eso se desarrollan aparte: son las que más enseñan.

#	Lo que el modelo afirmó	Lo que es cierto en 6.18.33.2	Cómo se detectó e impacto evitado
A1	`.llseek = no_llseek` en `file_operations`	Eliminado del kernel en 6.12; no hay coincidencias en el árbol	Inspección antes de compilar. Error de compilación. Representativo: aparece en casi todo ejemplo de driver de caracteres anterior a 2024
A2	`touch .scmversion` corrige el sufijo `+` de versión local	`scripts/setlocalversion` de 6.18 ya no consulta ese archivo; hay que desactivar la detección de git	Empíricamente: se aplicó y la versión no cambió. **Más peligrosa que A1**: la recomendación no produce ningún error, simplemente no hace nada, y el fallo aparece al cargar el módulo con un mensaje que no apunta a su causa
A3	El árbol de fuentes no corresponde al kernel en ejecución, porque `GCC_ASM_GOTO_OUTPUT_BROKEN` exige `GCC_VERSION < 110500`	El bloque tiene **tres** cláusulas `default`, y la tercera (`>= 130000 && < 130300`) sí incluye a GCC 13.2	Imprimiendo el bloque completo. **Grave**: habría llevado a abandonar la ruta del módulo cargable, el 40 % de la nota, por un problema inexistente. No fue un error de conocimiento sino de **evidencia insuficiente**
A4	«gcc-13 sí soporta el atributo `counted_by`», según una prueba encadenada con `&&`	GCC 13 emite warning: attribute directive ignored y **termina con código 0**; el test real del Kconfig usa `-Werror=attributes`	Al leer el test real. Habría descartado gcc-13, que era la solución correcta. **No basta con ejecutar una prueba: hay que replicar las condiciones exactas**

#	Lo que el modelo afirmó	Lo que es cierto en 6.18.33.2	Cómo se detectó e impacto evitado
A5	`CONFIG_CC_HAS_COUNTED_BY` es la causa del desajuste de CRC, presentado como hecho	La hipótesis **resultó correcta**, pero se afirmó antes de verificar que `genksyms` procesa atributos en su gramática	No la detectó nada: se acertó. Se registra porque es el modo de fallo más difícil de ver —una afirmación causal confiada y no verificada es indistinguible de una verificada hasta que resulta falsa

A6 — El `errno` de `mprotect`: `EPERM` escrito de memoria

Al documentar la PoC A3 se afirmó que `mprotect(PROT_WRITE)` sobre un VMA sin `VM_MAYWRITE` devuelve `EPERM`. Es el error que dice «no tienes permiso» en la intuición de cualquiera, y encajaba con la narrativa. La afirmación se propagó a **seis documentos**, incluido el bloque de «salida esperada» del `README`, que llegó a mostrar `Operation not permitted` como si fuera una captura real. `mm/mprotect.c:959-963`:

```
/* newflags >> 4 shift VM_MAY% in place of VM_% */
if ((newflags & ~(newflags >> 4)) & VM_ACCESS_FLAGS) {
        error = -EACCES;
        break;
}
```

El núcleo contrasta cada permiso pedido contra su `VM_MAY*` y devuelve **`EACCES`**. No lo detectó una relectura: lo detectó la PoC, porque `mg_poc.py` compara el `errno` real contra el esperado en vez de limitarse a comprobar que la llamada falló, y marcó A3 como `INCONCLUSIVE`. El cliente en C nunca lo habría delatado, porque imprime `strerror(errno)` sin compararlo con nada: mostraba el error correcto y el documento que lo acompañaba decía otra cosa. **Una PoC que solo verifica que el ataque falla no distingue entre fallar por el motivo previsto y fallar por otro.**

A7 — Suponer que el módulo ve las banderas que pidió el usuario

La PoC A2 daba por hecho que `mmap(MAP_SHARED)` sobre `/dev/mmapguard` tenía que ser rechazado por `mg_mmap()`, que comprueba `vma->vm_flags & VM_SHARED`. La primera ejecución marcó **FAIL**: el mapeo fue concedido. `mm/mmap.c:462-464`, dentro de `do_mmap()`:

```
vm_flags |= VM_SHARED | VM_MAYSHARE;
if (!(file->f_mode & FMODE_WRITE))
        vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
```

El dispositivo se abre en `O_RDONLY`, así que **el núcleo retira `VM_SHARED` y `VM_MAYWRITE` antes de llamar a `f_op->mmap`**. El módulo nunca ve la bandera que creía filtrar, y lo que concede es un mapeo privado de solo lectura, tan restringido como el del caso `MAP_PRIVATE`. No era un fallo de seguridad, pero deja ver algo que ningún repaso del módulo habría mostrado: **la comprobación de `VM_SHARED` de `mg_mmap()` es inalcanzable con un descriptor de solo lectura.** Para ejercitarla hay que abrir en `O_RDWR`, que es lo que hace ahora el tercer caso de A2. **El conjunto de banderas que recibe un `f_op->mmap` no es el que pidió el proceso: es el que el núcleo dejó después de aplicar sus propias reglas.**

A8 — El instrumento mintió: `r--s` no significa `VM_SHARED`

La corrección de A7 se apoyó en `/proc/self/maps`: si el núcleo retiraba `VM_SHARED`, el mapeo debía aparecer como `r--p`. La PoC corregida volvió a marcar **FAIL**, ahora con «el VMA quedó compartido (`r--s`)». Dos intentos, dos veredictos equivocados sobre un módulo que nunca estuvo mal. `fs/proc/task_mmu.c:454`:

```
seq_putc(m, flags & VM_MAYSHARE ? 's' : 'p');
```

El cuarto carácter depende de **`VM_MAYSHARE`**, no de `VM_SHARED`, y `do_mmap()` retira `VM_MAYWRITE | VM_SHARED` dejando `VM_MAYSHARE` intacto. Un mapeo ya degradado a privado sigue mostrándose como `s`. El instrumento correcto es la línea `VmFlags:` de `/proc/self/smaps` (`fs/proc/task_mmu.c:1152-1159`), con un mnemónico por bandera: `rd`, `wr`, `sh` = `VM_SHARED`, `mw` = `VM_MAYWRITE`, `ms` = `VM_MAYSHARE`. Comprobado sobre un archivo ordinario, sin el módulo:

Mapeo	`maps`	`VmFlags` de `smaps`
`O_RDONLY` + `MAP_SHARED`	`r--s`	`me mr ms rd sd` — **sin `sh`, sin `mw`**
`O_RDWR` + `MAP_SHARED`	`r--s`	`me mr ms mw rd sd` **`sh`**
`O_RDONLY` + `MAP_PRIVATE`	`r--p`	`me mr mw rd sd`

Las dos primeras filas son indistinguibles en `maps` y describen VMA con permisos distintos. La tercera añade el matiz que sostiene la pregunta 8: un mapeo **privado** desde un `fd` de solo lectura conserva `mw`, y por eso admite `PROT_WRITE` por copia en escritura, mientras que el compartido lo pierde.

Las tres últimas comparten estructura: una abstracción de usuario (`errno`, las banderas de `mmap`, la columna de `maps`) se dio por equivalente a la estructura de kernel subyacente, y no lo era. La verificación no termina al mirar la salida de una herramienta: termina al comprobar **qué imprime esa herramienta exactamente**, que también es código del kernel y tiene sus propias reglas.

Sobre el ejemplo de `CR0` que cita el enunciado

El enunciado menciona, como alucinación típica, deshabilitar el bit de protección contra escritura de `CR0` sin control de concurrencia. **No se produjo en esta sesión**, porque la evaluación arquitectónica descartó el hook en la syscall table antes de solicitar código para ella. Se deja constancia de que no ocurrió en lugar de atribuirse una detección que no tuvo lugar.

4. Hallazgos de verificación proactiva

No son alucinaciones: son hechos comprobados contra el árbol antes de escribir código, de modo que nunca llegaron a convertirse en errores. Son la evidencia de que el procedimiento se aplicó.

#	Hecho verificado	Archivo y línea	Por qué importaba
V1	`class_create()` recibe un solo argumento desde 6.4	`include/linux/device/class.h:226`	El ejemplo habitual pasa `THIS_MODULE`
V2	`vm_flags` es `const` en una unión; hay que usar `vm_flags_set/clear`	`include/linux/mm.h:808,815`	`vma->vm_flags |= …` no compila en 6.x
V3	`security_add_hooks()` no está exportada	`security/`	Descarta un LSM desde un módulo cargable
V4	`kallsyms_lookup_name` y `sys_call_table` no exportadas	`kernel/kallsyms.c`	Sustenta el descarte del hook de syscalls
V5	El gancho LSM `file_mmap` **no existe**; el real es `mmap_file`	`include/linux/lsm_hook_defs.h:199`	El propio enunciado usa el nombre antiguo
V6	`put_unaligned_be64()` vive en `<linux/unaligned.h>`	`include/linux/unaligned.h`	`<asm/unaligned.h>` ya no está en el árbol
V7	`.page_mkwrite` disponible en `vm_operations_struct`	`include/linux/mm.h:638`	Habilita la defensa en profundidad del plano 1
V8	6.18 añade `f_op->mmap_prepare`, excluyente con `.mmap`	`include/linux/fs.h:2285,2314,2385`	Punto de intercepción posterior al corte de conocimiento habitual
V9	`do_mmap()` retira `VM_SHARED` y `VM_MAYWRITE` sin `FMODE_WRITE`	`mm/mmap.c:462-464`	**A posteriori**, por el FAIL de A2 (ver A7)
V10	`mprotect` devuelve `-EACCES`, no `-EPERM`	`mm/mprotect.c:959-963`	**A posteriori**, por el INCONCLUSIVE de A3 (ver A6)
V11	La `s` / `p` de `maps` depende de `VM_MAYSHARE`	`fs/proc/task_mmu.c:454`	**A posteriori**, por el segundo FAIL de A2 (ver A8)
V12	`smaps` sí separa `sh`, `ms` y `mw`	`fs/proc/task_mmu.c:1152-1159`	Es el instrumento con que la PoC afirma el estado del VMA

Por qué la distinción entre las secciones 3 y 4 es el hallazgo central

Las alucinaciones de la sección 3 se detectaron **porque alguien fue a mirar**. Ninguna se delató sola: `no_llseek` habría fallado en compilación, pero `.scmversion` no produjo error alguno y la conclusión errónea sobre el árbol era internamente coherente y plausible.

A6, A7 y A8 son la excepción que confirma la regla, y por eso V9–V12 están marcadas como verificaciones a posteriori. No las encontró nadie leyendo: las encontró el banco al ejecutarse, porque afirma valores concretos —el `errno` exacto, las banderas reales del VMA— en lugar de comprobar que el ataque fracasa. Una prueba que solo exigiera «esto debe fallar» habría dado PASS en los tres casos y habría dejado intactos los seis documentos equivocados. **El oráculo no estaba en el árbol del kernel: estaba en exigirle a la prueba que fuera específica.** Y conviene no suavizarlo: el banco marcó FAIL **dos veces seguidas** sobre un módulo que siempre estuvo bien. Un veredicto rojo no prueba un defecto en el código: prueba un **desacuerdo** entre el código y lo que alguien creía del código, y el defecto puede estar en cualquiera de los dos lados.

El riesgo real de usar asistentes de IA en programación de sistemas no son las afirmaciones obviamente falsas, que el compilador filtra, sino las **plausibles y silenciosas**: APIs que existieron, soluciones que funcionaron hace tres versiones, y razonamientos correctos sobre datos incompletos.

5. Procedimiento de validación manual y compilación

Verificación contra el árbol. Se clonó `linux-msft-wsl-6.18.33.2`, que corresponde exactamente al kernel en ejecución, y **ninguna afirmación sobre una API se dio por válida de memoria**. **Compilación sin advertencias:** exigido por el punto 4.1, módulo y herramientas compilan con `-Wall -Wextra` sin una sola. **Validación empírica:** ningún componente se dio por bueno hasta ejecutarlo —carga con `insmod`, verificación en `dmesg`, banco completo y contadores—. **Validación cruzada de las mediciones:** tres salvaguardas detalladas en 4.2, el consumo obligatorio del checksum, once muestras con mediana y caché caliente verificada con `majflt = 0`.

Compilación contra los headers de la distribución utilizada. Requisito explícito del enunciado y el punto que más esfuerzo costó. El kernel de WSL2 lo compiló Microsoft con GCC 13.2; al regenerar la configuración con el GCC 15.2 del sistema, `make olddefconfig` autodetectó capacidades distintas —entre ellas

`CONFIG_CC_HAS_COUNTED_BY` —, lo que altera el texto de las cabeceras que `genksyms` convierte en CRC de símbolo. El módulo compilaba y su vermagic coincidía, pero `insmod` lo rechazaba con `Invalid module format` y, en `dmesg`, «disagrees about version of symbol module_layout». El diagnóstico exigió comparar ambas configuraciones símbolo a símbolo (19 diferencias, de las que solo una era relevante), confirmar que los CRC se calculan con `genksyms` sobre texto preprocesado y no con `gendwarfksyms` sobre DWARF, y verificar que la gramática de `genksyms` procesa atributos. La solución fue instalar GCC 13 y recompilar: el CRC de `module_layout` cambió de `0x5bc3c700` a `0xadc86883`. Quedó una diferencia residual —el trío de opciones `asm goto` — que **se predijo inocua y se documentó como predicción, no como hecho**, razonando que protegen cuerpos de función y macros, no declaraciones de tipos, que es lo único que `genksyms` procesa. Se confirmó al cargar el módulo.

3.3 Estrategia Arquitectónica Seleccionada

> Kernel objetivo: `6.18.33.2-microsoft-standard-WSL2` (x86_64)

1. Modelo de amenaza

Atacante asumido. Proceso local sin privilegios —sin `root`, sin `CAP_SYS_ADMIN`, sin `CAP_SYS_MODULE` — que puede ejecutar código arbitrario en Ring 3, abrir descriptores, invocar `mmap()`, `mprotect()`, `madvise()` y `msync()`, y lanzar procesos e hilos concurrentes. **Activo protegido:** un archivo confidencial cifrado en reposo que un proceso autorizado necesita leer por acceso por puntero, sin pagar el costo de `read()`.

Objetivo del enunciado	Significado operativo
Aislamiento de memoria	El texto en claro nunca reside completo en RAM ni es legible por un proceso no autorizado
Prevención de modificaciones	Un mapeo escribible sobre el activo se deniega antes de que exista la PTE
Auditoría de accesos	Todo intento, exitoso o denegado, produce un evento verificable

Fuera de alcance. Atacante con `root` (puede `rmmod` el módulo); ataques físicos o por DMA; canales laterales de microarquitectura; compromiso del hipervisor.

2. Alternativas evaluadas y criterio de descarte

Las afirmaciones de esta sección **no son de memoria**: cada una se verificó por inspección del árbol `linux-msft-wsl-6.18.33.2`, que es el del kernel en ejecución, y las citas con archivo y línea están en la tabla de verificaciones de la sección 3.2 (V2–V5, V7).

2.1 LKM con hook en la System Call Table — DESCARTADA. Cuatro razones independientes, cada una suficiente: la tabla no es alcanzable por vía legítima (`sys_call_table` no exportada, y `kallsyms_lookup_name()` tampoco desde 5.7, de modo que obtener la dirección exige un `kprobe` sobre la propia función, un rodeo cuyo único propósito es evadir una decisión deliberada de los mantenedores); exige **limpiar el bit WP de `CR0`**, textualmente el ejemplo de práctica insegura que cita el enunciado; es propensa a carreras, porque durante la reescritura otra CPU puede estar ejecutando esa misma syscall; y en x86_64 choca con el despacho por envoltorios `__x64_sys_*(struct pt_regs *)`, con CFI y con `lockdown`.

2.2 LSM propio desde un módulo cargable — INVIABLE. `security_add_hooks()` no está exportada y la inicialización del framework es `__init`, con la tabla fijada en compilación. No existe forma soportada; la ruta exigiría recompilar e instalar un kernel por cada iteración de desarrollo.

2.3 eBPF solo con `fentry`/`kprobes` — INSUFICIENTE. Observan pero **no pueden denegar**: su valor de retorno no altera el flujo. Sirven para auditoría, no para prevención, y el enunciado exige prevención. Se conserva como plano de observabilidad, no como mecanismo de control.

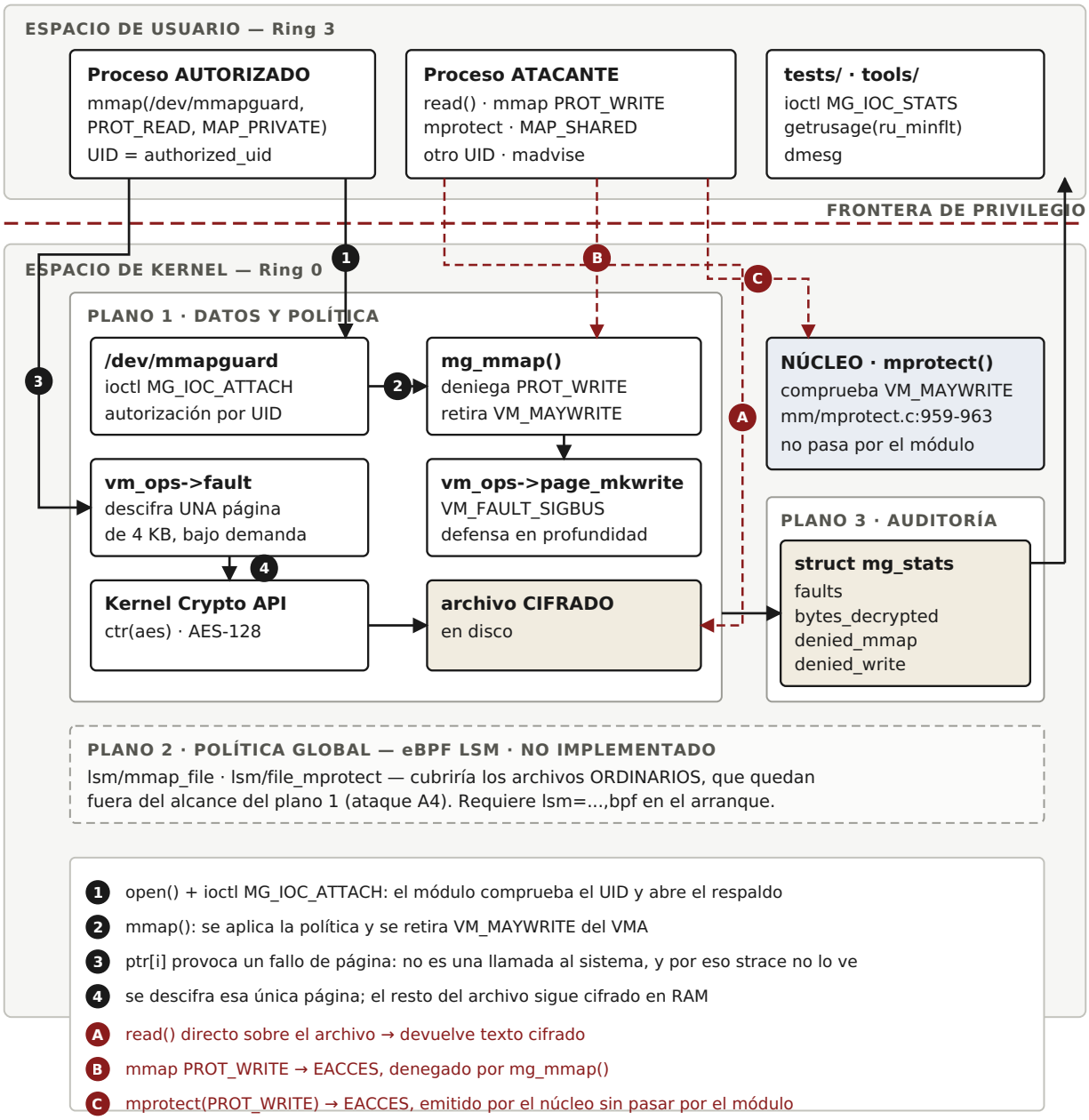
3. Arquitectura seleccionada: tres planos

Ningún mecanismo único cubre los tres objetivos. La arquitectura asigna **un plano por objetivo**:

Objetivo	Plano	Mecanismo	Artefacto	Estado
Aislamiento	**Datos**	LKM con `vm_ops->fault` y descifrado por página	`mmapguard.ko`	**Implementado**
Prevención	**Política**	Rechazo en `mg_mmap()` + `vm_flags_clear(VM_MAYWRITE)`	`mmapguard.ko`	**Implementado**
		eBPF LSM sobre `mmap_file` y `file_mprotect` (cobertura global)	`mg_lsm.bpf.c`	No implementado
Auditoría	**Observabilidad**	`struct mg_stats` vía `ioctl` + `getrusage`	`mmapguard.ko`, `mg_bench`	**Implementado**

El plano 2 se desdobla porque la prevención **sí se logró**, pero dentro del plano 1 y con alcance limitado a lo que pasa por `/dev/mmapguard`. La variante eBPF LSM, que la extendería a cualquier archivo del sistema, quedó condicionada a un requisito de arranque que este entorno no satisface. **Lo que falta no es el control, es su alcance.**

3.1 Diagrama



Arquitectura de `mmapguard`: los tres planos, la frontera de privilegio y los tres ataques del banco de pruebas.

3.2 Plano 1 — Datos: `mmapguard.ko`

Dispositivo de caracteres `/dev/mmapguard`. El flujo es `open()`, luego `ioctl(MG_IOC_ATTACH, struct mg_attach_req {path, flags})`, que compara el UID efectivo del llamante con el parámetro `authorized_uid` del módulo y prepara el respaldo del archivo cifrado, y finalmente `mmap()` sobre el descriptor. La clave no viaja en el `ioctl`: es un parámetro del módulo (`key_hex`), de modo que el espacio de usuario nunca la manipula.

Decisión central: `vm_ops->fault` en lugar de `remap_pfn_range()`. El paso 4 del repositorio de referencia usa `remap_pfn_range()`, que programa de golpe todas las PTE del rango y obliga a mantener **el archivo completo descifrado en RAM** durante toda la vida del mapeo. Con el fault handler se descifra **una página de 4 KB a la vez, y solo cuando el proceso la toca**: la ventana de exposición en memoria física baja de «todo el archivo» a «las páginas realmente usadas». Eso reconvierte la paginación por demanda, que en el material de referencia es una optimización, en un **control de minimización de exposición**. Medido: un cliente que lee los primeros 64 bytes de un archivo de 2 páginas deja 1 página descifrada y 4.096 bytes en claro en RAM, no 8.192.

Segunda decisión: el mapeo es estrictamente de solo lectura. El cifrado es AES-CTR con el bloque contador derivado del índice de página, que es lo que permite descifrar cada página de forma independiente. Pero CTR es un cifrado de flujo: reutilizar clave y nonce sobre dos textos planos distintos revela su XOR. Permitir escritura obligaría a re-cifrar cada página sucia con el mismo nonce en cada `msync()`, incurriendo sistemáticamente en esa falla. La prevención de escritura se implementa entonces en dos capas:

1. **`vm_flags_clear(vma, VM_MAYWRITE)` en el alta del mapeo.** Retirar esa bandera hace que sea **el propio kernel** quien rechace tanto `PROT_WRITE` en `mmap()` como una escalada posterior con `mprotect()`. No se intercep-

ta `mprotect` en ninguna parte: se usa la misma maquinaria descrita en la pregunta 2 para que la escalada sea imposible por construcción.

2. **`.page_mkwrite` como defensa en profundidad.** Es el único callback donde el kernel recupera el control ante la primera escritura sobre una página limpia ya mapeada —el «punto crítico de vulnerabilidad» del enunciado—. Con la capa anterior en su sitio no debería ejecutarse nunca; si lo hace, significa que algo burló la primera barrera, y deniega con `VM_FAULT_SIGBUS`.

El soporte de lectura-escritura con detección de integridad exige un modo AEAD (ChaCha20-Poly1305 o AES-GCM) con contador de versión por página y un área de metadatos para los tags: queda como trabajo futuro. Se prefirió un cifrado de solo lectura correcto frente a uno de lectura-escritura con una debilidad criptográfica conocida.

3.3 Plano 2 — Política: `mg_lsm.bpf.c` (no implementado)

Programas eBPF adjuntos a `SEC("lsm/mmap_file")` y `SEC("lsm/file_mprotect")`, que retornan `-EPERM` para denegar. `mmap_file` denegaría `PROT_WRITE | MAP_SHARED` sobre inodos protegidos y cualquier mapeo desde UIDs no autorizados, **antes de que exista el VMA**; `file_mprotect` cierra la evasión clásica de mapear benignamente y escalar después, sin la cual el primer gancho es trivialmente rodeable. La política viviría en un `BPF_MAP_TYPE_HASH` indexado por `(dev, inode)`, de modo que se cambia sin recargar nada. Su ventaja decisiva sobre el plano 1 es que protege **archivos reales del sistema de archivos**, no solo lo que pase por nuestro dispositivo.

Requisito de arranque. `CONFIG_BPF_LSM=y` está activo, pero `CONFIG_LSM` no incluye `bpf` y `/proc/cmdline` no trae parámetro `lsm=`. Hay que añadir `bpf` a la lista vía `kernelCommandLine` en `.wslconfig`. Sin eso los ganchos existen pero **no son adjuntables**.

3.4 Plano 3 — Observabilidad

La auditoría reside en el propio módulo, en una `struct mg_stats` **por sesión** que el usuario recupera con `ioctl(MG_IOC_STATS)`: `faults` (páginas materializadas por `.fault`), `bytes_decrypted` (la exposición real en RAM), `denied_mmap` y `denied_write`. A eso se suman los contadores que el kernel ya lleva por proceso, leídos con `getrusage()` en `mg_bench`.

Un hallazgo que condicionó el diseño de este plano: la instrumentación tenía que vivir dentro del kernel, porque **un fallo de página no es una llamada al sistema** (§4.4 de la sección 4.2).

4. Matriz de compensaciones

Alternativa	Cobertura	Profundidad de inspección	Sobrecosto	Riesgo de panic	Viable aquí
Hook de syscall table	Global	Solo argumentos de la syscall	Bajo	**Alto**	Sí, pero insegura
LKM con `vm_ops->fault`	Solo su dispositivo	**Máxima: página a página**	**Alto**: un fallo por página	Medio	**Sí**
LSM compilado en el kernel	Global	Alta	Bajo	Bajo	No: exige recompilar
eBPF LSM	**Global**	Media: decide al crear el VMA	**Muy bajo**: una vez por `mmap`	**Nulo**	Sí, con `lsm=…,bpf`
eBPF `fentry` / `kprobe`	Global	Solo observación	Muy bajo	Nulo	Sí

El eje del compromiso: la profundidad de inspección se paga en fallos de página. El plano 1 ve el contenido de cada página, pero cuesta un `#PF` y una operación criptográfica por página tocada. El plano 2 decide **una sola vez**, en el alta del VMA, con costo marginal cero, pero nunca ve el contenido. Son complementarios, no sustitutos, y de ahí la decisión de combinarlos. La hipótesis cuantitativa que valida la Parte III es que el sobrecosto del plano 1 escale con las páginas efectivamente tocadas y no con el tamaño del archivo.

5. Banco de pruebas: ataques a demostrar

El banco vive en `tests/run_pocs.sh` (orquestación, verificación del entorno y resumen) y `tests/mg_poc.py` (los ataques, por `ctypes` contra libc, para controlar las banderas exactas de `mmap` / `mprotect` / `madvise` y leer el `errno` exacto de cada rechazo); `tools/mg_test` queda como cliente de referencia en C. Cada PoC se corresponde con una pregunta de la Parte I.

Los siete ataques, sus resultados medidos y el análisis de los contadores están en la sección 4.2; aquí solo las dos decisiones de diseño del banco que no son del patrón «se intenta, se deniega».

A4 demuestra primero que la evasión existe. Mostrar que `/dev/mmapguard` rechaza el mapeo no prueba nada por sí solo: hay que saber que el ataque funciona en otro sitio. La PoC abre `demo/secreto.txt` en `O_RDONLY`, comprueba que el núcleo **concede** `PROT_WRITE | MAP_PRIVATE`, altera los ocho primeros bytes en memoria y verifica por `sha256` que el archivo en disco quedó intacto —el escenario exacto de la pregunta 8— y solo entonces repite el ataque contra el dispositivo.

A5 mide en vez de argumentar. La fase 1 lanza un hijo aislado que intenta escribir y comprueba que muere con `SIGSEGV`: la señal no se puede capturar, de modo que la muerte del hijo es el resultado positivo. La fase 2 martillea `MADV_DONTNEED` desde cuatro hilos mientras el principal reverifica el digest completo, y mide `ru_minflt` con `getrusage()` para probar que las PTE se descartaban de verdad; sin ese dato, un martilleo que no descarta nada «pasaría» la prueba sin haber probado nada. Esa medición obligó a corregir este documento, que antes afirmaba que tras el descarte la página «vuelve a fallar y se redescifra»: lo segundo es falso, y la consecuencia para el modelo de amenaza está en el §2.4 de la sección 4.2.

Trazabilidad con la Parte I. Las respuestas 4 y 7 justifican el descarte de la sección 2 y la elección del plano 2; las respuestas 1, 3, 8, 9 y 10 fundamentan el diseño del plano 1.

4.1 Implementación: código, compilación y pruebas de concepto

El código completo está en *`src/`*, *`tools/`* y el *`Makefile`*. Esta sección explica las decisiones de implementación; las instrucciones de uso están en el *`README.md`* y los resultados en 4.2.

1. El contrato con el espacio de usuario

Toda la interfaz cabe en una cabecera compartida, `src/mmapguard_uapi.h`, compilada tanto por el módulo como por las herramientas:

```
struct mg_attach_req {
        char  path[MG_PATH_MAX];   /* ruta absoluta del archivo cifrado */
        __u64 flags;               /* reservado */
};
struct mg_stats {
        __u64 file_size, faults, bytes_decrypted, denied_mmap, denied_write;
};
#define MG_IOC_ATTACH   _IOW('G', 1, struct mg_attach_req)
#define MG_IOC_STATS    _IOR('G', 2, struct mg_stats)
```

El flujo del cliente son tres pasos: `open("/dev/mmapguard")`, `ioctl(MG_IOC_ATTACH)` indicando qué archivo cifrado exponer, y `mmap()` sobre ese descriptor; la auditoría se recupera con `ioctl(MG_IOC_STATS)`. **El dispositivo no decide qué archivo se protege; lo decide el cliente autorizado:** el módulo no lleva una lista de rutas codificada, de modo que la misma instancia sirve para cualquier archivo cifrado con la clave cargada.

2. Estado por sesión y su tiempo de vida

```
struct mg_session {                         /* src/mmapguard.c:105-114 */
        struct file                 *backing;   /* archivo cifrado en disco     */
        loff_t                       size;
        unsigned long                npages;
        struct page                 **pages;     /* cache de paginas descifradas */
        struct crypto_sync_skcipher *tfm;       /* transformacion AES-CTR       */
        struct mutex                 lock;      /* protege pages[]              */
        struct mg_stats              stats;
};
```

La sesión se asigna en `mg_open()` y se cuelga de `filp->private_data`. Hay un detalle de corrección que no es evidente: el VMA conserva una referencia a `vma->vm_file`, de modo que **la sesión sobrevive aunque el proceso cierre el descriptor después de mapear** —el patrón habitual `fd = open(); mmap(fd); close(fd);`—. Si se hubiera ligado al proceso en lugar de al `struct file`, ese cierre habría dejado el VMA con un puntero colgante y el primer fallo posterior habría sido un oops. La liberación ocurre en `mg_release()`, que además imprime el resumen de auditoría a `dmesg`, de modo que queda rastro aunque el cliente nunca consulte `MG_IOC_STATS`.

3. Derivación del IV: lo que hace viable la paginación por demanda

```
static void mg_build_iv(u8 iv[MG_IV_BYTES], pgoff_t pgoff)   /* :162-168 */
{
        u64 block = (u64)pgoff * (PAGE_SIZE / MG_AES_BLOCK);

        memset(iv, 0, MG_IV_BYTES);
        put_unaligned_be64(block, iv + 8);
}
```

AES-CTR cifra generando un flujo de bloques a partir de un contador. Si el contador de la página n se puede **calcular** en lugar de derivar del estado anterior, cada página se descifra de forma independiente y en cualquier orden. Como una página de 4.096 bytes son exactamente 256 bloques AES de 16 bytes, el contador de la página `pgoff` es `pgoff × 256`, en los 8 bytes bajos del IV en orden de red. Sin esta propiedad habría que descifrar secuencialmente desde el principio del archivo para llegar a una página cualquiera, y el acceso aleatorio de `mmap()` —su razón de ser— sería imposible. `tools/mg_encrypt.c` replica la derivación con OpenSSL, de modo que el ci-

frado en usuario y el descifrado en kernel coinciden por construcción. `put_unaligned_be64` requiere `<linux/unaligned.h>`; `<asm/unaligned.h>`, la que aparece en casi todo el material publicado, ya no existe.

4. El manejador de fallos: el corazón del diseño

`mg_vma_fault()` (`src/mmapguard.c:230-305`) se invoca una vez por página, la primera vez que el proceso la toca:

Paso	Qué hace	Por qué así
1	Valida `pgoff < npages`	Un fallo fuera de rango devuelve `VM_FAULT_SIGBUS`, no accede a memoria ajena
2	Toma `s->lock`	`pages[]` es compartido entre hilos del mismo mapeo
3	Si la página ya existe, `get_page()` y listo	Evita descifrar dos veces: el segundo acceso cuesta un fallo menor normal
4	`alloc_page(GFP_KERNEL | __GFP_ZERO)`	Marco propio del módulo, inicializado a cero
5	Calcula `to_read`, truncando en la última página	Un archivo que no es múltiplo de 4 KB no debe leer basura más allá del final
6	`kmap_local_page()` + `kernel_read()`	Lee el texto cifrado directamente en el marco
7	`mg_decrypt_page()` descifra **en el sitio**	Sin copia intermedia: no hay un segundo búfer con texto plano
8	Registra en `stats` y entrega con `vmf->page`	El contador se actualiza bajo el mismo `lock`

Dos decisiones del camino de error merecen mención. Si la lectura o el descifrado fallan, la página se borra con `memzero_explicit()` **antes** de devolverla al asignador: un marco con texto plano parcial no vuelve al sistema. Y se devuelve `VM_FAULT_SIGBUS`, no `VM_FAULT_OOM`, para que el proceso reciba una señal y no se interprete como presión de memoria. El descifrado usa `SYNC_SKCIPHER_REQUEST_ON_STACK` y una `scatterlist` de un solo elemento que apunta a la propia página, con el mismo origen y destino; la transformación se pide por nombre genérico, `crypto_alloc_sync_skcipher("ctr(aes)", 0, 0)`, para que la API de criptografía del kernel elija la implementación acelerada del hardware.

5. La política: aplicada antes de que exista una sola PTE

`mg_mmap()` (`src/mmapguard.c:475-529`) es el único punto donde se decide, y se ejecuta en el alta del VMA:

```
if (!mg_caller_is_authorized()) {          /* UID efectivo */
        s->stats.denied_mmap++;
        pr_warn("mmapguard: MMAP DENEGADO para UID %u (PID %d)\n", ...);
        return -EPERM;
}
if (vma->vm_flags & VM_WRITE)  { s->stats.denied_mmap++; return -EACCES; }
if (vma->vm_flags & VM_SHARED) { s->stats.denied_mmap++; return -EACCES; }

vm_flags_clear(vma, VM_MAYWRITE);
vm_flags_set(vma, VM_DONTEXPAND | VM_DONTDUMP);

vma->vm_ops          = &mg_vm_ops;
vma->vm_private_data = s;
```

`vm_flags_clear(vma, VM_MAYWRITE)` es la línea más importante del módulo. No rechaza nada por sí misma: retira del VMA la capacidad de llegar a ser escribible. A partir de ahí es el propio núcleo quien rechaza cualquier `mprotect(PROT_WRITE)` posterior, por cualquier camino que lo solicite, sin que el módulo intercepte nada. La política deja de ser una lista de comprobaciones —que obliga a haber enumerado todos los puntos de entrada— y pasa a ser un **invariante del estado del VMA**.

Las otras dos banderas cierran huecos concretos: `VM_DONTEXPAND` impide crecer el mapeo con `mremap()`, y `VM_DONTDUMP` lo excluye de los volcados de memoria, que de otro modo escribirían el texto plano a disco. Sobre la mutación: `vma->vm_flags` es `const` dentro de una unión en los kernels 6.x, y hay que usar `vm_flags_set/clear`, que además toman el bloqueo de escritura del VMA (`vma_start_write()`). La asignación directa `vma->vm_flags |= …`, que aparece en todo el material anterior a 2023, no compila.

6. Gestión de memoria y residuos

`mg_release_pages()` (`src/mmapguard.c:197-216`) borra explícitamente cada marco antes de liberarlo con `memzero_explicit(page_address(s->pages[i]), PAGE_SIZE)` y no con `memset()`, porque el compilador puede eliminar un `memset` cuyo resultado no se lee: es exactamente el mismo modo de fallo que invalidó el banco de pruebas del repositorio de referencia, aplicado a un borrado de seguridad en lugar de a una medición.

Limitación declarada. La caché `pages[]` no tiene mecanismo de reclamo: una página descifrada permanece en memoria hasta que se cierra la sesión. Para un archivo recorrido por completo el consumo llega al tamaño del archivo —16 MB en nuestras mediciones— y la exposición de texto plano crece de forma monótona con las páginas tocadas. Es aceptable para el caso de uso que motiva el diseño (archivos de secretos, leídos por partes) e inaceptable para archivos grandes recorridos íntegramente. La corrección natural es registrar un shrinker o acotar la caché con una política de expulsión; no está implementado.

7. Compilación sin advertencias

```
obj-m          += mmapguard.o
mmapguard-objs := src/mmapguard.o
KDIR           ?= /lib/modules/$(shell uname -r)/build
ccflags-y      := -I$(src)/src -Wall -Wextra
```

El enunciado exige compilación libre de advertencias, y `-Wall -Wextra` está en `ccflags-y`, no como recomendación externa: el módulo y las tres herramientas compilan limpios desde cero. El `Makefile` ofrece además `make tools`, `make demo` (regenera los datos de prueba de forma determinista), `make test`, `make informe`, `make clean`, `make distclean` y `make help`.

La restricción que no es del Makefile sino del entorno: hay que compilar con `CC=gcc-13`, porque el kernel de WSL2 lo compiló Microsoft con GCC 13.2 y `CONFIG_MODVERSIONS=y` exige que los CRC de los símbolos coincidan. El episodio completo está en la sección 3.2 §5.2.

8. Herramientas de espacio de usuario

Herramienta	Qué hace	Nota de implementación
`mg_encrypt.c`	Cifra un archivo con el esquema que espera el driver	OpenSSL `EVP_aes_128_ctr()`, un IV por página con la derivación del §3
`mg_test.c`	Cliente de referencia: A1–A3 y la auditoría	Imprime el `errno` de cada intento, no solo éxito o fallo
`mg_bench.c`	Comparativa a tres bandas con `getrusage()`	11 repeticiones y mediana; imprime todos los checksums
`tests/run_pocs.sh`	Orquesta el banco y resume el resultado	Verifica el entorno, correlaciona con `dmesg`, agrega códigos de salida
`tests/mg_poc.py`	Las PoC A0–A5, contra libc por `ctypes`	Sin compilar nada: banderas y `errno` exactos de `mmap` / `mprotect` / `madvise`

El banco está en dos piezas y en dos lenguajes, y la división no es arbitraria. El enunciado pide scripts en Bash o Python, y además hay trabajo que en C habría costado mucho más: provocar un `SIGSEGV` deliberado en un hijo aislado, martillear `madvise` desde varios hilos y leer el `errno` exacto de cada combinación de banderas. `ctypes` da acceso directo a libc sin pasar por el `Makefile`, de modo que el banco se modifica y se reejecuta sin recompilar. Que `mg_bench` imprima los checksums no es cosmético: es lo que impide que `-O2` elimine los bucles de medición por código muerto.

9. Pruebas de concepto

El banco completo se ejecuta con `./tests/run_pocs.sh --load` (o `make test TESTFLAGS=--load`, que carga el módulo y lo descarga al terminar), cubre A0–A5 y **pasa 8 de 8**; la salida íntegra queda en `tests/last-run.log`. Los resultados medidos, el mapeo de cada PoC a su pregunta de la Parte I y el análisis de los contadores están en la sección 4.2.

4.2 Medición de Impacto y Análisis de Resultados

1. Entorno y metodología

Elemento	Valor
Kernel	`6.18.33.2-microsoft-standard-WSL2`, x86_64, 5 niveles de paginación
CPU / RAM	Intel Core i5-12450H, 12 hilos / 7,6 GiB
Compilador del módulo	GCC 13.4.0 (misma rama que el kernel, por CRC de MODVERSIONS)
Implementación de AES	`ctr-aes-vaes-avx2` (acelerada por hardware, prioridad 600)
Conjunto de datos	16.777.285 bytes = 4.097 páginas

El cifrado está acelerado por hardware, dato necesario para interpretar los resultados: el sobrecosto que se mide **no** está dominado por el costo criptográfico.

Se comparan tres formas de acceder al mismo contenido: `read()` en bloques de 4 KB sobre el archivo en claro, `mmap()` ordinario sobre el archivo en claro, y `mmapguard` sobre el archivo cifrado. Tres decisiones metodológicas, cada una contra un defecto concreto:

1. **La caché de páginas se calienta antes de medir**, para medir el costo del mecanismo y no el del disco. Lo confirma que los fallos mayores (`majflt`) son 0 en todas las corridas.
2. **Once muestras por método, y se reporta la mediana.** Una sola muestra de la latencia de `mmap()` quedó dominada por el ruido del planificador: dos corridas consecutivas de la misma configuración dieron 43,2 µs y 3,2 µs. Sin repetición, ese número no significa nada.

3. **Los checksums se imprimen.** Si el resultado del bucle no se consume, GCC con `-O2` elimina el bucle por código muerto y el método parece infinitamente rápido. Es el defecto del benchmark del paso 5 del repositorio de referencia (sección 3.1 §6).

Los tres métodos producen **checksums idénticos** en todas las configuraciones, lo que verifica que el descifrado por página es correcto.

2. Resultados funcionales

Banco completo (`./tests/run_pocs.sh --load`) sobre `secreto.enc` , 4.764 bytes = 2 páginas. **8 de 8 PASS**; la salida íntegra queda en `tests/last-run.log` .

PoC	Ataque	Resultado medido	Mecanismo que lo bloquea	Preg.
A0	`MG_IOC_ATTACH` desde el UID 65534	`EPERM` , `denied_mmap = 1`	`mg_caller_is_authorized()`	4
A1	Lectura directa con `read()`	Ilegible: fracción imprimible 0,41	Cifrado en reposo	5
OK	Lectura por el mapeo protegido	Los 4.764 bytes coinciden con el claro	— (control positivo)	1, 5
A2	`mmap(PROT_WRITE)` ; `mmap(MAP_SHARED)`	`EACCES` ; ver §2.2	`mg_mmap()` y el kernel	2
A3	`mprotect(PROT_WRITE)` tras mapeo legítimo	`EACCES` , `denied_mmap` sin cambio	**El kernel**, por `VM_MAYWRITE` retirado	3
A4	`O_RDONLY` + `PROT_WRITE` + `MAP_PRIVATE`	Concedido sobre archivo ordinario; `EACCES` sobre el dispositivo	El kernel lo permite; `mg_mmap()` lo cierra	8
A5	`MADV_DONTNEED` contra escritura	`SIGSEGV` ; 0 discrepancias en 11.229 relecturas	La MMU, por el bit R/W de la PTE	9

El control positivo no es decorativo: sin él, un módulo que denegara absolutamente todo pasaría el resto del banco. Verifica el mapeo **completo** contra el texto en claro, no un prefijo.

2.1 La asimetría de los contadores

`denied_mmap` quedó en **1**, no en 2, en la sesión que ejecuta A2 y A3 juntos: el intento de `mprotect` **nunca llegó al módulo**, lo rechazó el propio kernel. Es la confirmación empírica de que retirar `VM_MAYWRITE` en el alta del VMA hace innecesario interceptar `mprotect()` . Y `denied_write` quedó en **0**: `page_mkwrite` nunca llegó a ejecutarse, porque la primera barrera resultó suficiente —la defensa en profundidad no se activó, que es justamente lo que debe ocurrir cuando la capa anterior funciona—. Los contadores son **por sesión**, no globales: viven en el `struct mg_session` que cuelga de cada descriptor abierto, de modo que las cifras no se acumulan entre pruebas.

2.2 A2: el kernel llega antes que el módulo

`mmap(MAP_SHARED)` sobre un descriptor abierto en `O_RDONLY` **es concedido**, y no es un fallo de la política: `do_mmap()` retira `VM_SHARED` y `VM_MAYWRITE` cuando el archivo no tiene `FMODE_WRITE` (`mm/mmap.c:462-464`), de modo que `mg_mmap()` nunca llega a ver la bandera que filtra. Medido sobre el VMA resultante:

```
maps dice 'r--s'
smaps VmFlags: dd de me mr ms rd sd      <- sin 'sh', sin 'mw'
mprotect(PROT_WRITE) sobre el: DENEGADO con Permission denied
```

El mapeo concedido es privado y de solo lectura, tan restringido como el del caso `MAP_PRIVATE` . Con el descriptor en `O_RDWR` , `VM_SHARED` sobrevive y entonces sí responde el módulo: `EACCES` con `denied_mmap = 1` . Conviene no leer el `r--s` de `maps` como evidencia de nada: ese carácter depende de `VM_MAYSHARE` , no de `VM_SHARED` (`fs/proc/task_mmu.c:454`). El episodio está documentado como A7 y A8 en la sección 3.2.

2.3 A4: la evasión de `O_RDONLY` existe, y se mide

Sobre `secreto.txt` abierto en `O_RDONLY` el kernel **concede** `PROT_WRITE | MAP_PRIVATE` . La PoC altera los primeros ocho bytes en memoria — `NOMINA C` → `AAAAAAAA` — y comprueba por `sha256` que el archivo en disco no cambió: el escenario exacto de la pregunta 8. El mismo ataque sobre `/dev/mmapguard` devuelve `EACCES` . El contraste con A2 lo explica una sola bandera: un mapeo **privado** desde un descriptor de solo lectura conserva `mw` (`VM_MAYWRITE`), porque puede hacerse escribible por copia en escritura; el **compartido** la pierde.

2.4 A5: la carrera no tiene precondición, y el descarte no redescifra

Fase 1: un proceso hijo intenta escribir en el mapeo protegido y **muere con** `SIGSEGV` , por el bit R/W de la PTE y sin que el módulo ejecute una sola instrucción. Sin escritura posible no hay nada que serializar contra el descarte, que es la precondición que describe la pregunta 9. Fase 2, cuatro hilos martilleando `MADV_DONTNEED` durante 2 segundos contra un lector que reverifica el digest completo:

Magnitud	Valor
Descartes (`madvise`)	21.615
Relecturas completas del mapeo	11.229

Magnitud	Valor
Discrepancias de contenido	**0**
Fallos de página menores del proceso (`ru_minflt`)	**+6.665**
`faults` y bytes descifrados por el driver	**+0**

Los +6.665 fallos son la prueba de que el martilleo funcionaba: las PTE se descartaban y rematerializaban de verdad. Sin esa cifra, un martilleo que no descartara nada habría «pasado» la prueba sin demostrar nada.

Las dos últimas filas son el hallazgo: **el descarte no provoca un solo descifrado nuevo**. El fallo rematerializado encuentra la página en `s->pages[pgoff]` y devuelve una referencia a la misma página en claro (`src/mmapguard.c:252-259`), sin volver a descifrar y sin incrementar `faults` (`src/mmapguard.c:297`). La consecuencia para el modelo de amenaza es concreta: **la ventana de exposición del texto en claro es la vida de la sesión, no la de la PTE.** `MADV_DONTNEED` retira la traducción del proceso, no la copia descifrada del módulo, que solo se borra en `mg_release()` con `memzero_explicit()`. Estrechar esa ventana —descartar páginas en claro por presión de memoria o por temporizador— queda como trabajo futuro.

3. Sobrecosto según el patrón de acceso

Mediana de 11 muestras; `minflt` = fallos de página menores. El archivo es el mismo en los tres regímenes: solo cambia el patrón de acceso.

Régimen	Método	Pared (s)	CPU kernel (s)	minflt	`mmap()` (µs)
Secuencial	`read()` 4 KB	0,00200	0,00209	0	—
4.097 págs.	`mmap()` normal	0,00038	0,00038	257	8,1
	`mmapguard`	0,00677	0,00677	**4.097**	3,1
Salto de 8	`read()` 4 KB	0,00022	0,00020	0	—
513 págs.	`mmap()` normal	0,00026	0,00024	257	3,4
	`mmapguard`	0,00077	0,00069	**513**	2,4
Disperso (64)	`read()` 4 KB	0,00002	0,00002	0	—
65 págs.	`mmap()` normal	0,00005	0,00005	65	1,2
	`mmapguard`	0,00010	0,00010	65	1,6

Patrón	Páginas tocadas	Sobrecosto vs. `mmap()`	Archivo descifrado
Secuencial	4.097	**17,60x**	100 %
Salto de 8	513	**2,93x**	12,5 %
Disperso (64)	65	**1,94x**	**1,6 %**

4. Análisis: de dónde viene realmente el costo

4.1 El sobrecosto escala con páginas tocadas, no con el tamaño del archivo. De 17,60x a 1,94x sobre el mismo archivo de 16 MB, variando únicamente el patrón de acceso. Es la propiedad que justifica la arquitectura: el precio se paga por página efectivamente usada.

4.2 El costo dominante NO es el cifrado, es la ausencia de fault-around. La columna `minflt` lo explica: en acceso secuencial, `mmap()` ordinario resuelve 4.097 páginas con solo **257 fallos**, porque el kernel implementa `filemap_map_pages()` y puebla hasta 16 PTE por fallo. `mmapguard` implementa únicamente `.fault`, de modo que paga **un fallo por página**. La razón entre ambos es 15,9x, prácticamente idéntica al sobrecosto observado de 17,60x. La prueba está en el régimen disperso: con salto de 64 páginas el fault-around no aporta nada —ambos ejecutan 65 fallos— y el sobrecosto cae a 1,94x. Ese residuo sí es el costo real de `kernel_read()` más AES por página. Un modelo de dos puntos separa ambas componentes: leer y descifrar una página cuesta ≈ **0,8 µs** (del régimen disperso, donde los fallos coinciden) y un fallo de página adicional cuesta ≈ **0,8 µs** (del residuo en el secuencial). Es decir, en acceso secuencial **aproximadamente la mitad del sobrecosto es evitable** y no tiene nada que ver con la seguridad.

4.3 La latencia de la llamada `mmap()` está por debajo del ruido. Los valores medidos —3,1 contra 8,1 µs, 2,4 contra 3,4 µs, 1,6 contra 1,2 µs— no mantienen un orden consistente: en dos de las tres configuraciones `mmapguard` resulta más rápido que `mmap()` ordinario, lo cual es físicamente imposible dado que hace trabajo adicional. La conclusión correcta no es que sea más rápido, sino que **el sobrecosto de la política aplicada en el alta del VMA no es medible con esta instrumentación**: queda sepultado bajo la varianza del planificador. Eso confirma cuantitativamente la predicción de la matriz de compensaciones de la sección 3.3: una decisión tomada una sola vez tiene costo marginal despreciable.

4.4 `strace` es el instrumento equivocado, y eso mismo es el hallazgo. Perfil de syscalls de una corrida completa:

```
% time     seconds  usecs/call     calls    syscall
 97.46    0.484200          10     45069    pread64
  0.08    0.000381          12        30    mmap
```

Las 45.069 llamadas `pread64` (11 repeticiones × 4.097 páginas) provienen **exclusivamente del método `read()`**; los métodos basados en memoria mapeada aportan 30 llamadas `mmap` y nada más. Las 4.097 operaciones de descifrado por repetición que ejecuta `mmapguard` **son invisibles para `strace`**, porque un fallo de página no es una llamada al sistema. Es el «punto crítico de vulnerabilidad» del enunciado visto desde la instrumentación: si la defensa no puede observarse con herramientas de syscalls, el atacante tampoco. Medir mecanismos basados en `mmap` exige `getrusage()`, `ftrace` o `perf`, nunca `strace`.

5. Optimización identificada y no implementada

Implementar `vm_operations_struct->map_pages` permitiría poblar varias PTE por fallo, igual que `filemap_map_pages()`. Según el modelo del §4.2 eso reduciría el sobrecosto secuencial de 17,60x a aproximadamente **9x**, sin debilitar ninguna propiedad de seguridad: las páginas seguirían descifrándose individualmente y bajo demanda, solo que agrupando la instalación de entradas.

6. Limitaciones de estas mediciones

1. Se tomaron en WSL2, sobre un hipervisor: la varianza es mayor que en hardware nativo y los valores absolutos no son comparables con los de un servidor dedicado. Las razones entre métodos, medidas en las mismas condiciones, sí son significativas.
2. No se midió con `perf`, porque en WSL2 exige compilarlo desde `tools/perf` del propio árbol.
3. El modelo de dos puntos del §4.2 es una estimación de primer orden, no una descomposición exacta.
4. No se midió el caso de caché fría: todas las corridas tienen `majflt = 0`. Un escenario con lectura real de disco cambiaría el equilibrio **a favor** de `mmapguard`, porque el costo de E/S dominaría sobre el del descifrado.

4.2 (b) Justificación Teórica: por qué la intervención debe ocurrir en la capa del kernel

1. El criterio correcto no es el rendimiento

Es tentador justificar Ring 0 por velocidad: menos cambios de contexto, menos copias, `copy_to_user()` eliminado. El argumento es cierto para `read()` contra `mmap()`, y es el que desarrolla el material de referencia del curso, pero **no es el que sostiene esta solución**. De hecho la medición apunta en contra: `mmapguard` es 17,60x más lento que `mmap()` ordinario en acceso secuencial. Si el criterio fuera el rendimiento, habríamos perdido.

El criterio correcto es el clásico del **monitor de referencia**: un mecanismo de seguridad vale si es (a) inviolable —el adversario no puede modificarlo ni desactivarlo—, (b) invocado siempre —no existe camino al recurso que lo esquive, la mediación completa de Saltzer y Schroeder— y (c) suficientemente pequeño para verificarse. La tesis:

> Para el objetivo que pide el enunciado —controlar el acceso a las páginas de un archivo proyectado en memoria— las soluciones en espacio de usuario fallan la condición (b) por una razón **estructural, no de implementación**: el evento que hay que mediar no es una llamada al sistema.

2. El argumento decisivo: un fallo de página no es una syscall

Todo lo que el espacio de usuario puede interceptar es una **frontera de llamada al sistema**: una biblioteca interpuesta sustituye el envoltorio de libc, `seccomp` filtra la entrada a la syscall, `ptrace` se detiene en ella, un demonio FUSE recibe una petición cuando el VFS se la enruta. El acceso a una página mapeada no cruza ninguna de esas fronteras: es una **excepción de hardware**. La MMU encuentra la PTE sin el bit present, la CPU genera un page fault, y el manejador lo resuelve íntegramente en Ring 0. El proceso nunca ejecutó una instrucción que un monitor en usuario pudiera observar.

No es una deducción teórica: está medido. `strace` sobre el cliente de `mmapguard` registra **30 llamadas a `mmap` y ninguna señal de las 4.097 páginas descifradas** (§4.4 de la sección anterior). El trabajo de seguridad completo es invisible al instrumento que observa syscalls, y lo que es invisible a `strace` es invisible, por la misma razón, a cualquier monitor que viva en espacio de usuario. De ahí el corolario que ordena toda la comparación:

> El espacio de usuario puede mediar el **alta del mapeo** (`mmap()`), que es una syscall. No puede mediar el **acceso a la página**, que no lo es. Y mediar solo el alta es una comprobación única: después, el mapeo queda gobernado por bits de PTE que el monitor no ve ni puede consultar.

3. Examen de las alternativas en espacio de usuario

3.1 Cifrado dentro de la propia aplicación. No media nada: protege el dato en reposo y nada más. Una vez descifrado, el texto plano vive en memoria anónima del proceso, legible por cualquier otro proceso del mismo

UID vía `ptrace()` o `/proc/pid/mem`, volcable a un core dump y susceptible de ir a swap. Falla (a) y (b) a la vez, y exige modificar cada aplicación: la mediación se pierde por omisión en cuanto un programa no colabora.

3.2 Interposición de biblioteca (`LD_PRELOAD`). Falla de forma trivial y conocida: una syscall se puede emitir **directamente**, con `syscall(SYS_mmap, …)` o con la instrucción `syscall` en ensamblador, sin tocar libc; además `LD_PRELOAD` no existe para binarios estáticos y el cargador lo ignora en binarios setuid. Falla (b) por construcción, y (a) con igual claridad: el código interpuesto reside en el **mismo espacio de direcciones que el adversario**, que puede resolver el símbolo original con `dlsym(RTLD_NEXT, …)` o simplemente desmapear al interpositor. Un monitor de referencia situado dentro del dominio que vigila no es un monitor de referencia.

3.3 `seccomp-bpf` y `SECCOMP_RET_USER_NOTIF`. Es la alternativa más fuerte en el plano de las syscalls y merece tratarse en serio, porque **sí puede denegar**: la decisión la aplica el kernel, aunque el filtro lo defina el usuario. Primer límite: un filtro `seccomp` **no puede desreferenciar punteros**, y la documentación del árbol lo dice sin ambigüedad —«BPF programs may not dereference pointers which constrains all filters to solely evaluating the system call arguments directly», `Documentation/userspace-api/seccomp_filter.rst:26-27` —. El filtro ve los registros, no la memoria: puede denegar todo `mmap` con `PROT_WRITE`, pero **no puede resolver el descriptor al archivo concreto**, de modo que es incapaz de expresar una política por archivo. Segundo límite: `SECCOMP_RET_USER_NOTIF` delega en un supervisor de usuario que sí puede leer la memoria del objetivo, pero el mismo documento advierte del TOCTOU que eso reintroduce (`:284-290`); y el supervisor sigue siendo un proceso de usuario —matable, `ptrace`-able, planificable a conveniencia del adversario—. Y el límite que lo cierra: `seccomp` media **syscalls**, de modo que puede mediar la entrada a `mmap()` y jamás el acceso a la página.

3.4 FUSE: sistema de archivos cifrado en espacio de usuario. Es la alternativa seria, la que usan `gocryptfs` y `EncFS`, y sería deshonesto despacharla: **sí logra mediación completa para la E/S de archivos**, porque es el VFS del kernel el que enruta las peticiones al demonio, y no hay forma de esquivarla emitiendo syscalls directas. Su fallo es específico de `mmap`, y se verifica en el árbol (`fs/fuse/file.c:2323-2328`):

```
static const struct vm_operations_struct fuse_file_vm_ops = {
        .close          = fuse_vma_close,
        .fault          = filemap_fault,
        .map_pages      = filemap_map_pages,
        .page_mkwrite   = fuse_page_mkwrite,
};
```

`.fault = filemap_fault`. **FUSE sirve los fallos de página desde la caché de páginas genérica, no desde el demonio.** El demonio se consulta cuando la página no está en caché; una vez poblada, todo acceso posterior se resuelve dentro del kernel sin ninguna llamada ascendente. De ahí tres consecuencias: el demonio **no puede observar ni denegar el segundo acceso a una página**, de modo que lo que media es el poblado de la caché y no el uso; la residencia del texto plano la gobierna la política de reclamo de la caché y no la de seguridad, así que no se puede expresar «olvida esta página en cuanto se use»; y el texto plano **pasa por el espacio de direcciones del demonio**, que es un proceso de usuario. A esto se suma el costo: cada falta en caché son como mínimo dos cambios de contexto y dos copias, frente a nuestro descifrado en el propio camino del fallo.

Alternativa	¿Inviolable (a)?	¿Mediación completa del acceso (b)?	Política por archivo	Claro en proceso de usuario
Cifrado en la aplicación	No	No media nada	Sí	**Sí**
`LD_PRELOAD`	No	No: syscall directa lo esquiva	Sí	**Sí**
`seccomp-bpf`	Sí (lo aplica el kernel)	No: un fallo no es syscall	**No**: no desreferencia punteros	n/a
`seccomp` + `USER_NOTIF`	No: supervisor en usuario	No: un fallo no es syscall	Sí, con TOCTOU	n/a
FUSE	No: demonio en usuario	**No**: la caché sirve los fallos	Sí	**Sí**
LKM con `vm_ops->fault`	Sí	**Sí: está en el camino del fallo**	Sí	**No**

4. Tres propiedades que solo se obtienen en Ring 0

4.1 La política como invariante, no como intercepción. Un monitor en usuario tiene que **interceptar y decidir** en cada punto de entrada, lo que obliga a haberlos enumerado todos: `mmap`, `mprotect`, `mremap`, `madvise`, `process_vm_writev`… Olvidar uno es un agujero. `mmapguard` no intercepta `mprotect` en ninguna parte: limpia `VM_MAYWRITE` del VMA, de modo que es **el propio núcleo** quien rechaza cualquier intento posterior de obtener permiso de escritura, por cualquier camino que lo pida. Está demostrado empíricamente —`denied_mmap` marcó **1 y no 2**—, y la corrección no depende de que hayamos previsto el ataque: depende de un invariante que el kernel comprueba en todas las rutas relevantes, **incluidas las que no conocemos**.

4.2 El texto plano nunca reside en un proceso de usuario, ni la clave. La clave se entrega como parámetro de módulo con permiso `0` —`module_param(key_hex, charp, 0)`, `src/mmapguard.c:85` —, y permiso `0` significa que **no se crea entrada en sysfs**: no es legible desde `/sys/module/mmapguard/parameters/`, vive en memoria de ker-

nel y ningún proceso de usuario la tiene jamás. En cualquier esquema de la sección 3 la clave está necesariamente en el espacio de direcciones de un proceso —la aplicación, el interpositor o el demonio FUSE—. Y la exposición del texto plano está acotada por diseño, no por la política de reclamo del kernel (sección 3.3 §3.2).

4.3 Granularidad de página en el camino del fallo. El único lugar desde el cual se puede decidir por página y en el momento del acceso es el camino del fallo, y ese camino está en Ring 0 por definición arquitectónica. No es una preferencia de implementación: es dónde el hardware entrega el control.

5. El costo de la mediación completa, cuantificado

La sección 2 explica por qué hay que estar en el camino del fallo; la medición explica lo que eso cuesta, y el dato más interesante es la **simetría con FUSE**. Nótese que `fuse_file_vm_ops` incluye `.map_pages = filemap_map_pages`: FUSE obtiene fault-around gratis —16 páginas por fallo, `fault_around_pages = 65536 >> PAGE_SHIFT` en `mm/memory.c:5591` — precisamente **porque delega los fallos a la caché de páginas**, que es exactamente lo que le cuesta la mediación por acceso. Ahí está el intercambio, en una frase:

> FUSE compra fault-around a cambio de renunciar a mediar cada acceso. `mmapguard` compra la mediación de cada acceso a cambio de renunciar al fault-around.

Y eso es, literalmente, el 17,60x. **No estamos pagando por AES: estamos pagando por mediar.** La confirmación es que el costo es proporcional a la mediación que se consume y no un impuesto fijo: con salto 64, donde ambos esquemas hacen 65 fallos, cae a **1,94x** y solo se descifra el 1,6 % del archivo. El mecanismo cobra por página mediada.

6. Lo que el espacio de usuario hace mejor

Una comparación en la que una de las partes no tiene ninguna ventaja no es una comparación, es un argumento de venta. Estas son reales y pesan.

Dimensión	Ventaja del espacio de usuario
Aislamiento de fallos	Un error en un demonio FUSE mata un proceso. Un error en `mmapguard.ko` es un oops o un panic: se lleva el sistema. Es la desventaja más grave y no tiene mitigación dentro del enfoque.
Portabilidad	El episodio del CRC de `MODVERSIONS` es la evidencia: el módulo está atado a una versión de kernel **y a un compilador**. Un binario FUSE cruza versiones sin recompilar.
Tamaño de la base confiable	Añadir ~600 líneas al TCB, en C, sin red de seguridad, es un costo de seguridad real que hay que descontar del beneficio.
Costo de desarrollo y claves	Sin libc, sin coma flotante, pila limitada, depuración por `dmesg`; y un HSM, PKCS#11 o un KMS en red son naturales en usuario y penosos en un módulo.

Esto explica también por qué el plano 2 de la arquitectura es **eBPF LSM** y no más código de módulo: eBPF es la síntesis moderna de las dos columnas —mediación en Ring 0, en los ganchos LSM, pero con un **verificador** que elimina el riesgo de panic y con política recargable sin tocar el kernel—.

7. En qué sentido la capa del kernel es óptima

No en sentido absoluto. La afirmación defendible es condicional, y estas son sus cuatro condiciones:

1. **El evento a mediar es el acceso a una página, no la petición de mapeo.** Si bastara con mediar el alta, `seccomp` o FUSE serían preferibles por aislamiento de fallos y portabilidad.
2. **El modelo de amenaza incluye procesos del mismo UID, con `ptrace` y `/proc/pid/mem`.** Si el adversario fuera externo, cifrar en reposo en la aplicación bastaría.
3. **La exposición del texto plano en RAM es parte de lo que hay que acotar.** Es el requisito que descarta a FUSE y a `remap_pfn_range()`, y el que convierte la paginación por demanda de optimización en control de seguridad.
4. **Se acepta pagar el sobrecosto proporcional a las páginas tocadas.** Para un archivo de secretos leído por partes al arrancar, 17,60x secuencial es irrelevante; para un flujo de gran volumen es inaceptable, y la respuesta correcta sería cifrado a nivel de bloque, o `fscrypt`.

Bajo esas condiciones —que son las del enunciado— la intervención en la capa del kernel no es la mejor de varias opciones viables: es **la única que satisface la mediación completa**. Las demás no pierden por margen, pierden por construcción, porque median una frontera que el evento protegido no cruza.